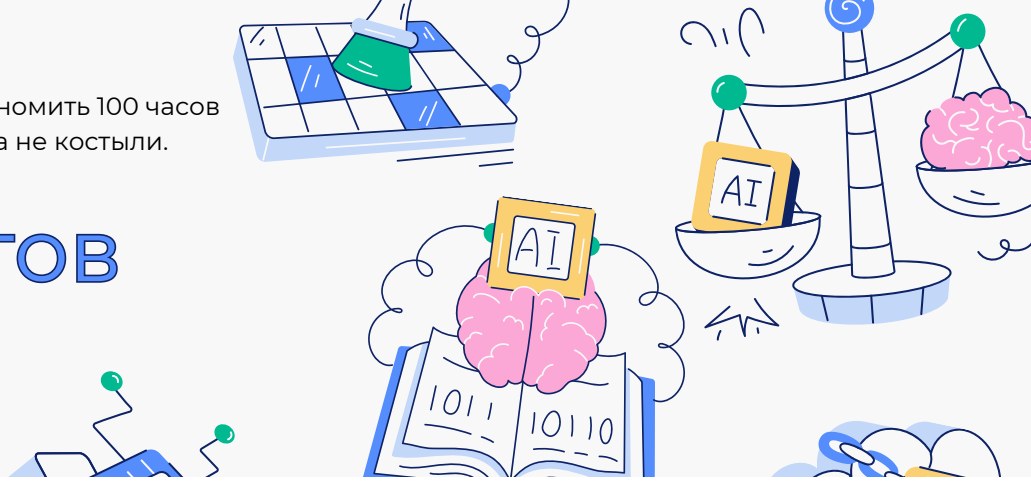
Библиотека вайб-кодера: как сэкономить 100 часов разработки и писать архитектуру, а не костыли.

50 промптов



Почему большинство разработчиков разочаровываются в ИИ?

Они используют Cursor или Claude как умный Google. Пишут: «*Сделай мне форму логина*», а потом часами дебажат нерабочий спагетти-код.

Вайб-кодинг работает иначе. Здесь ты не пишешь код руками — ты управляешь ИИ-агентами как Tech Lead. А для этого нейросети нужно ставить жесткие рамки, давать правильный контекст и мыслить системами.

В этом документе я собрал 50 рабочих промптов, которые использую каждый день для запуска проектов за выходные. Они разделены на 5 этапов разработки: от закладки архитектуры до сложных интеграций в n8n.

Скопируй их, адаптируй под свои задачи — автоматизируй и властвуй.

Раздел 1. Инициализация и Архитектура системы

Эти промпты задают фундамент. Используй их в самом начале, до написания первой строчки кода.

1. Генерация файла .cursorrules (Установка правил игры)

> «Я начинаю новый проект: [краткое описание, например: SaaS для учета времени с AI-аналитикой]. Мой стек: [Next.js, Tailwind, Supabase]. Напиши для меня подробный файл .cursorrules. Включи в него строгие архитектурные конвенции: как мы именуем файлы, как структурируем компоненты, как работаем с состоянием и базой данных. Цель — чтобы при дальнейшей генерации кода ты всегда следовал этим единым стандартам и писал чистый, предсказуемый код.»

2. Выбор оптимального стека под задачу

> «Выступи в роли Senior Software Architect. У меня есть идея MVP: [описание идеи и целевой аудитории]. Мои главные ограничения: бюджет равен нулю, запустить нужно за 14 дней, я работаю один. Предложи 3 варианта технологического стека (от no-code до full-stack). Для

каждого варианта распиши плюсы, минусы и скорость разработки. Сделай итоговую рекомендацию.»

3. Проектирование структуры базы данных (Схема)

«Мы определились со стеком, используем [PostgreSQL / Prisma]. Спроектируй схему базы данных для [название проекта]. Мне нужны таблицы для сущностей: [Пользователи, Заказы, Платежи, Подписки]. Распиши все связи (1-to-many, many-to-many), типы данных и индексы. Выдай результат в виде готового кода схемы [например, schema.prisma или SQL-дамп] и добавь краткое объяснение, почему ты выбрал такую структуру.»

4. Инициализация файловой структуры проекта

«@Codebase Опираясь на нашу архитектуру, сгенерируй дерево папок и файлов для стартового шаблона. Распиши текстом, за что будет отвечать каждая директория (например, /components/ui, /lib/utils, /app/api). После того как я утвержу структуру, напиши bash-скрипт или список команд для терминала, чтобы я мог создать все эти папки и пустые файлы за одну секунду.»

5. Проектирование API-контрактов

«Спроектируй REST API [или GraphQL] для нашего проекта. Напиши контракты для эндпоинтов, которые будут отвечать за [основной бизнес-процесс, например: создание новой кампании и получение аналитики]. Укажи точные URL, методы (GET/POST), структуру ожидаемого request body и примеры response body (включая обработку ошибок). Оформи это в формате Markdown-таблицы.»

6. Архитектура безопасности и авторизации

«Предложи оптимальный флоу авторизации для проекта. Требования: [например, вход через Telegram, поддержка JWT-токенов, разделение ролей на Admin и User]. Опиши пошагово, как токены будут передаваться между клиентом и сервером, где они будут храниться на фронтенде (cookies vs local storage) и как мы защитим API-роуты от несанкционированного доступа.»

7. Выбор стратегии управления состоянием (State Management)

«Мы используем [React/Vue]. Для нашего проекта [описание сложности интерфейса] предложи лучшую стратегию управления состоянием. Нужно ли нам тянуть тяжелый Redux, или обойдемся Zustand / Context API / React Query? Аргументируй выбор с точки зрения производительности и минимизации boilerplate-кода. Напиши пример инициализации стора для выбранного инструмента.»

8. Планирование интеграций со сторонними сервисами

«В проекте планируются интеграции с внешними API: [Stripe для оплат, OpenAI для генерации текста, n8n для автоматизаций]. Опиши архитектурный паттерн, как мы будем изолировать логику работы с этими API от остального кода. Напиши шаблон класса или сервиса (на [TypeScript/Python]), который будет безопасно делать запросы, обрабатывать лимиты (rate limits) и логировать ошибки.»

9. Архитектура деплоя и CI/CD

> «Спроектируй пайплайн деплоя для MVP. Исходный код хранится на GitHub. Я хочу, чтобы при пуше в ветку main проект автоматически тестировался и разворачивался на [Vercel / VPS]. Распиши пошагово: какие сервисы использовать, как безопасно передать переменные окружения (.env) и дай шаблон файла GitHub Actions для базового CI/CD.»

10. "Стресс-тест" придуманной архитектуры

> «@Codebase Проанализируй всю архитектуру, которую мы только что утвердили. Найди 3 самых слабых места (bottlenecks). Что сломается первым, когда в проект одновременно зайдут 10 000 пользователей? Что будет сложнее всего масштабировать? Предложи превентивные решения для каждой из этих проблем до того, как мы начнем писать код.»

Раздел 2: Написание логики и Backend

Базовая логика и CRUD-операции

11. Быстрая генерация CRUD-эндпоинтов

> «Сгенерируй полноценный CRUD (Create, Read, Update, Delete) для сущности [Название сущности, например: Заказы]. Напиши контроллеры, сервисы и роуты. Используй [Express/Fastify/Next.js API] и ORM [Prisma/Drizzle]. Обязательно вынеси бизнес-логику в отдельный сервисный слой (Service Layer), чтобы контроллеры оставались тонкими.»

12. Централизованная обработка ошибок

> «Создай глобальный middleware для перехвата и обработки ошибок в нашем API. Он должен ловить все необработанные исключения, стандартизировать формат ответа (например, { status: 'error', code: 400, message: '...' }) и не отдавать стектрейс (stack trace) на клиент в production-среде. Добавь кастомные классы ошибок (AppError, ValidationError).»

13. Валидация входящих данных

> «@Codebase Напиши схему валидации для эндпоинта создания [Пользователя/Проекта], используя библиотеку [Zod / Joi / Yup]. Схема должна проверять типы данных, обязательность полей, формат email и минимальную длину пароля. Интегрируй эту схему в контроллер так, чтобы при неверных данных клиент автоматически получал код 400 и список конкретных ошибок по каждому полю.»

14. Стандартизация пагинации и фильтрации

> «Реализуй логику пагинации и фильтрации для получения списка [Товаров/Пользователей]. Используй cursor-based пагинацию (или offset/limit, если база небольшая). Напиши утилиту, которая парсит query-параметры из URL (?limit=20&page=2&status=active) и безопасно преобразует их в запрос к базе данных.»

База данных и сложные запросы

15. Сложные агрегации данных

«Мне нужно написать сложный запрос к базе данных для дашборда аналитики. Нужно получить: общее количество активных пользователей за месяц, процентный прирост по сравнению с прошлым месяцем и топ-5 самых популярных действий. Напиши этот запрос, используя [Prisma / чистый SQL]. Оптимизируй его так, чтобы он выполнялся за один проход базы (без N+1 запросов).»

16. Написание и запуск миграций

«Я добавил новое поле isPremium в таблицу Пользователей и новую связанную таблицу Subscriptions. @Codebase сгенерируй скрипт миграции базы данных для [PostgreSQL]. Объясни пошагово, какими командами в терминале мне применить эту миграцию локально и как откатить её, если что-то пойдет не так.»

17. Изоляция транзакций

«Напиши логику проведения платежа и обновления баланса пользователя. Критически важно: оберни эту логику в транзакцию базы данных (DB Transaction). Если оплата прошла, но создание записи в логах упало — все изменения должны откатиться (rollback). Используй синтаксис [твоей ORM].»

Авторизация, Безопасность и Доступ

18. Role-Based Access Control (RBAC)

«Реализуй систему проверки прав доступа (middleware). У нас есть 3 роли: User, Manager, Admin. Напиши функцию-обертку (guard), которой я смогу оборачивать любые эндпоинты, например: requireRoles(['Admin', 'Manager']). Функция должна читать роль пользователя из JWT-токена или сессии и отдавать 403 Forbidden, если прав не хватает.»

19. Защита от брутфорса и Rate Limiting

«@Codebase Настрой Rate Limiting для нашего API. На эндпоинты авторизации (/login, /reset-password) установи лимит: не более 5 попыток в минуту с одного IP-адреса. На остальные API-роуты — 100 запросов в минуту. Используй [Redis / memory store]. При превышении лимита должен возвращаться статус 429 Too Many Requests.»

20. Безопасное хеширование и работа с секретами

«Напиши сервис для работы с паролями и токенами. Используй [bcrypt / argon2] для хеширования паролей перед сохранением в базу (укажи оптимальный salt rounds). Напиши функции генерации случайных токенов для сброса пароля и их верификации. Убедись, что ни один секретный ключ не захардкожен в коде, а берется из .env.»

Интеграции и фоновые задачи

21. Интеграция с внешним API (с обработкой лимитов)

«Напиши сервис для отправки данных в [Stripe / OpenAI / Telegram Bot API]. Реализуй паттерн Exponential Backoff: если внешнее API отвечает ошибкой 429 или 5xx, скрипт должен

подождать несколько секунд и повторить запрос, увеличивая время ожидания. Сделай максимум 3 попытки, после чего логируй критическую ошибку.»

22. Прием и обработка Webhooks

«Мне нужно принимать вебхуки от [указать сервис, например: n8n или платежной системы]. Напиши эндпоинт, который будет получать POST-запрос. Добавь логику валидации подписи (signature validation), чтобы убедиться, что запрос пришел именно от этого сервиса, а не от злоумышленника. После успешной валидации передай данные в очередь или фоновую задачу.»

23. Запуск фоновых задач (Cron Jobs)

«@Codebase Напиши скрипт для регулярной фоновой задачи (Cron Job). Задача: каждый день в 02:00 ночи находить всех пользователей, у которых завтра заканчивается подписка, и отправлять им email с напоминанием. Раздели логику на сам планировщик (cron) и функцию бизнес-логики, чтобы её можно было тестировать отдельно.»

Оптимизация и сложные алгоритмы

24. Реализация системы кэширования

«Наш эндпоинт /api/stats работает слишком медленно, так как делает тяжелые расчеты. Интегрируй [Redis / In-memory cache] для кэширования ответов. Установи TTL (time-to-live) на 5 минут. Напиши логику инвалидации кэша: если администратор обновляет данные через другой эндпоинт, кэш должен принудительно сбрасываться.»

25. Сложный парсинг и трансформация данных

«У меня есть массив сложных вложенных JSON-объектов (грязные данные), получаемых из внешнего источника. Напиши алгоритм на [Твой язык], который эффективно преобразует этот массив в плоскую структуру, очищает поля от HTML-тегов, приводит даты к формату ISO 8601 и удаляет дубликаты по полю id. Оптимизируй код для работы с большими массивами (десятки тысяч записей).»

Блок 3: Верстка и UI/UX (Frontend)

Дизайн в Код и Архитектура компонентов

26. Пиксель-в-пиксель из макета (Vision Prompt)

«[Прикрепить скриншот или макет из Figma] Выступи в роли Senior Frontend Developer. Сверстай этот интерфейс максимально близко к макету, используя [Next.js + Tailwind CSS]. Сразу разбей верстку на переиспользуемые изолированные компоненты (например: Header, Sidebar, AnalyticsCard). Не хардкодь данные: вынеси их в константы или прокидывай через props. Обеспечь семантическую HTML5-разметку.»

27. Создание UI-Kit'а и дизайн-системы

«Мы собираем собственную дизайн-систему. Напиши базовый компонент Button. Он должен поддерживать 4 визуальных варианта (Primary, Secondary, Outline, Ghost), 3 размера (sm, md, lg) и состояние disabled (задизейблено). Используй утилиты вроде [cva (class-variance-authority) и tailwind-merge] для элегантного управления комбинациями классов без конфликтов. Добавь JSDoc-комментарии для удобного автокомплита пропсов в IDE.»

28. Мобильная адаптивность (Mobile-First)

«@Codebase Этот компонент отлично выглядит на десктопе, но ломается на экранах телефонов. Перепиши его стили с подходом Mobile-First. Используй [Tailwind breakpoints: sm, md, lg], чтобы интерфейс плавно перестраивался. Добавь скрываемое "бургер-меню" для экранов меньше 768px и убедись, что все интерактивные элементы достаточно крупные для тапа пальцем (размер тап-зоны минимум 44x44px).»

Сложные интерфейсы и Состояния

29. Умная многошаговая форма (Wizard)

«Напиши сложную многошаговую форму для онбординга пользователей. В ней 3 шага: 1. Личные данные, 2. Настройки профиля, 3. Выбор тарифа. Реализуй управление состоянием между шагами с помощью [React Hook Form / VueUse]. Форма не должна пускать пользователя на следующий шаг, пока все обязательные поля текущего экрана не пройдут строгую валидацию. Добавь индикатор прогресса (Progress bar) сверху.»

30. Дашборды и Визуализация данных

«Создай UI-компонент аналитического дашборда. Реализуй адаптивную сетку (CSS Grid), в которой будут красиво располагаться 4 карточки с ключевыми метриками (KPI) и один большой график продаж за месяц. Для графика используй библиотеку [Recharts / Chart.js]. Сгенерируй реалистичные моковые (mock) данные, чтобы график не был пустым, и добавь всплывающие тултипы (tooltips) при наведении на точки.»

UX, Анимации и Вайб

31. Микро-интеракции (Добавляем вайба)

«Интерфейс работает, но выглядит слишком статично и "деревянно". Добавь микро-интеракции для всех кликабельных элементов, используя [Framer Motion / Tailwind transitions]. Карточки должны плавно приподниматься при наведении (hover) с мягкой тенью. Кнопки должны иметь эффект легкого вдавливания при клике (active). Сделай анимации быстрыми и отзывчивыми (длительность не более 150-200мс).»

32. Идеальный Dark Mode (Темная тема)

«Внедри поддержку системной Темной и Светлой темы (Dark/Light mode) для всего приложения. Используй CSS-переменные (или настройки Tailwind) для управления палитрой. Основной фон в темной теме — насыщенный zinc-900, а акцентный цвет везде замени на **коралловый**. Добавь переключатель тем (Theme Toggle) с плавной анимацией смены иконок (Солнце/Луна) и сохранением выбора пользователя в localStorage.»

33. Скелетная загрузка (Skeleton Loaders)

«Замени стандартный и скучный крутящийся спиннер на Skeleton Loaders (каркасную загрузку). Сверстай плейсхолдеры, которые точно повторяют структуру нашего контента (круг для аватара, прямоугольник для заголовка, три тонкие линии для текста). Добавь им приятную пульсирующую анимацию (shimmer effect). Настрой логику так, чтобы они показывались ровно до момента получения isSuccess от бэкенда.»

Оптимизация и Доступность

34. Аудит доступности (a11y)

«Проведи аудит доступности (accessibility) этого модального окна. Добавь необходимые aria-атрибуты, атрибут tabindex и поддержку полного управления с клавиатуры (фокус должен логично перемещаться по клавише Tab и не выходить за пределы окна, пока оно открыто). Убедись, что кнопка закрытия корректно отрабатывает по нажатию на 'Escape'.»

35. Фронтенд-рефакторинг (Убийца спагетти-кода)

«@Codebase Этот компонент разросся до 400 строк и стал абсолютно нечитаемым. Проведи жесткий рефакторинг. Вынеси всю сложную бизнес-логику и запросы к API в кастомные хуки (custom hooks). Разбей гигантский блок верстки на 3-4 мелких, логичных под-компонента. Убедись, что лишних ререндеров не происходит (используй memo или useCallback только там, где это математически оправдано).»

Раздел 4: Рефакторинг и Дебаггинг

Поиск и исправление ошибок (Дебаггинг)

В вайб-кодинге ИИ пишет код быстро, но иногда начинает «галлюцинировать» или городить костыли. Этот блок промптов нужен для того, чтобы заставить нейросеть саму искать свои ошибки, чистить спагетти-код и писать тесты. Это сэкономит десятки часов, когда проект внезапно перестает компилироваться.

36. Вдумчивый дебаггинг (Убийца логов)

«Я получаю следующую ошибку в консоли: [вставить текст ошибки]. Не пиши мне сразу готовый код для слепого копипаста. Сначала объясни простым языком, из-за чего концептуально возникла эта ошибка. Затем предложи 2 варианта решения: "быстрый фикс" и "архитектурно правильное решение". Я выберу вариант, и тогда мы его реализуем.»

37. Метод резиновой уточки (Rubber Duck Debugging)

«@Codebase Я не понимаю, почему эта функция не работает так, как задумано. Ожидаемое поведение: [описать, что должно быть]. Текущее поведение: [описать, что происходит по факту]. Пройдись по коду строка за строкой и объясни вслух, как меняются переменные на каждом шаге. Найди логическую ошибку в этом потоке (flow).»

38. Выход из "Dependency Hell" (Ад зависимостей)

«При попытке npm install [или yarn/pnpm] у меня возникают конфликты версий (peer dependencies) между [Пакет А] и [Пакет Б]. Проанализируй мой package.json. Предложи безопасный способ разрешить эти конфликты. Нужно ли мне откатить версию какого-то пакета, использовать npm i --legacy-peer-deps (объясни риски) или заменить одну из библиотек на современный аналог?»

39. Тотальный откат (Когда всё сломалось)

«Твой предыдущий код полностью сломал приложение. Забудь последние 3 шага, которые мы сделали. Давай вернемся к изначальной задаче: [напомнить задачу]. Похоже, наш подход был в корне неверным. Предложи 3 принципиально других архитектурных способа решить эту проблему, не затрагивая [указать модуль, который ломался].»

Очистка и Рефакторинг

40. Уничтожение мертвого кода

«Проанализируй этот файл. Найди и удали весь "мертвый код": неиспользуемые переменные, импорты, которые нигде не вызываются, закомментированные куски логики и недостижимые ветки if/else. Сделай код максимально сухим (DRY). В ответе кратко перечисли, что именно ты вырезал.»

41. Внедрение паттернов проектирования

«Этот класс/модуль превратился в "God Object" (объект, который знает и делает слишком много). Отрефактори его логику, применив подходящий паттерн проектирования (например, Strategy, Observer или Factory). Разнеси ответственность по разным файлам так, чтобы код соответствовал принципам SOLID (особенно Single Responsibility).»

42. Жесткая типизация (TypeScript Police)

«В этом проекте слишком много типов any и ts-ignore. @Codebase пройдись по файлу и замени все any на строгие интерфейсы (Interfaces) или типы (Types). Если функция возвращает сложный объект, опиши его структуру. Убедись, что параметры всех функций типизированы корректно.»

Производительность и Безопасность

43. Профилирование производительности (Big O Оптимизация)

«Эта функция отрабатывает слишком медленно при передаче в нее массива из 10 000 элементов. Текущая вычислительная сложность, скорее всего, O(n^2). Перепиши алгоритм так, чтобы снизить сложность до O(n) или O(n log n). Используй хэш-таблицы (Map/Set) или другие структуры данных для оптимизации поиска. Объясни, за счет чего достигнуто ускорение.»

44. Аудит безопасности (Поиск уязвимостей)

«Выступи в роли специалиста по кибербезопасности. Проведи аудит этого кода на предмет уязвимостей из списка OWASP Top 10 (например: SQL-инъекции, XSS, CSRF, небезопасное хранение данных). Если найдешь потенциальные дыры, покажи, как злоумышленник может их эксплуатировать, и перепиши код так, чтобы закрыть эти уязвимости.»

Тестирование

45. Генерация осмысленных тестов

> «Напиши Unit-тесты для этой функции, используя [Jest / Vitest]. Не пиши тесты только для "идеального сценария" (happy path). Обязательно покрой краевые случаи (edge cases): передача null, пустых массивов, отрицательных чисел или невалидных строк. Напиши моки (mocks) для любых внешних зависимостей (например, запросов к БД или API).»

Блок 5: Интеграции и Автоматизация (n8n, API)

---

n8n и Работа с данными

Этот блок — твоя козырная карта. Он показывает, как вайб-кодинг выходит за пределы IDE и превращается в реальный бизнес-процесс (то, что ты делаешь с n8n и AI-агентами).

Здесь мы заставляем нейросеть писать скрипты для автоматизаций, разбираться в чужих API и связывать сервисы между собой.

46. Идеальный JS-скрипт для Code Node (n8n)

> «Я собираю сценарий в n8n. Мне нужен JavaScript-код для ноды Code. На вход поступает массив JSON-объектов (items), содержащий данные о [например: лидах из Facebook Ads]. Напиши скрипт, который отфильтрует объекты, где поле phone пустое, затем преобразует все номера телефонов к единому международному формату (+X XXX XXX XX XX), и вернет новый массив объектов, готовый для передачи в следующую ноду. Учитывай специфику структуры данных в n8n (формат items[0].json).»

47. JSON-хирург (Маппинг сложных структур)

> «У меня есть два сервиса. Первый отдает данные в формате XML (или запутанном JSON), а второй (наша CRM) принимает только строгий JSON определенной структуры. [Вставить пример входящих данных] и [Вставить требуемую структуру]. Напиши эффективный скрипт парсинга и маппинга данных на [JavaScript/Python], который аккуратно переложит значения из первого формата во второй, обработав возможные null или отсутствующие поля без падения скрипта.»

API и Авторизация

48. "Взломщик" документации (OAuth2 и сложные API)

> «Мне нужно интегрировать [название малоизвестного сервиса или локальной платежки] в мой проект. Вот ссылка на их документацию API: [ссылка] (или вставленный текст доки). Изучи процесс авторизации. Объясни мне пошагово, как получить access-токен (какие ключи куда передавать), как часто его нужно обновлять (refresh token) и напиши готовую функцию-коннектор, которая будет делать аутентифицированные запросы к их эндпоинту /v1/data.»

49. Оркестратор ИИ-агентов (Связка LLM)

«Я строю систему из нескольких ИИ-агентов. Первый агент анализирует входящие запросы клиентов, второй — ищет информацию в базе знаний (RAG), третий — генерирует финальный ответ. Напиши логику "Оркестратора" (на Python или JS), который будет принимать запрос, определять его намерение (intent) и последовательно вызывать нужных агентов, передавая контекст от одного к другому. Добавь обработку ошибок: что делать, если второй агент завис или выдал галлюцинацию?»

Социальные сети и Маркетинг

50. Автоматизатор контента (Для соцсетей и рекламы)

«@Codebase Спроектируй архитектуру микросервиса (или сценария n8n) для автоматической генерации и публикации контента. Триггер: появление новой статьи в RSS-ленте. Сервис должен забрать текст статьи, передать его в [Claude/OpenAI] с промптом для написания вовлекающего поста для LinkedIn и Telegram (с разными форматами), сгенерировать картинку через внешний API и отправить готовые посты через [Telegram Bot API / LinkedIn API]. Напиши код для узловых точек этой связки.»

Умение писать код постепенно обесценивается. Ценность смещается в сторону тех, кто умеет ***проектировать системы, автоматизировать рутину и управлять ИИ-агентами****.*

Эти 50 промптов — лишь инструмент. Настоящая магия начинается тогда, когда вы соединяете их в единый рабочий процесс.

Хотите увидеть, как эти подходы применяются на живых проектах? Как мы собираем SaaS-продукты, настраиваем агентов в n8n и монетизируем вайб-кодинг?

Следующий шаг: *Присоединяйтесь к нашему комьюнити предпринимателей и разработчиков.*

Telegram-канал (здесь вся практика): https://t.me/startup_14day
Instagram: https://www.instagram.com/timur.yessenov/
LinkedIn: linkedin.com/in/timur-yessenov
Проекты: Founder @ Nocodia & AI Vibe Club